

Introduction

Browser fingerprinting is a technique that aims to uniquely identify a user on a website by using the technical characteristics of their browser and track them across different time periods or web sites. It relies on the analyses of over 40 different attributes (and their combination) such as screen resolution, system configuration, or rendering behavior.

Unlike cookies, fingerprinting cannot simply be blocked without affecting core browser functionalities and is mostly undetectable. As a result, defenses must balance privacy and usability.

In this context, the Brave web browser proposes a set of protection against fingerprinting. With this paper, based on Brave's official documentation, we will re-implement three of these protections to test their effectiveness. We decided to not directly test the protections features live on Brave for reproducibility matters.

This report is the work of 3 people, that's why it's a little longer than mentioned in the guidelines.

First, we will look into how the plugins list announced by the browser is protected.

Secondly, we will dive into the uniformization of attributes values based on the brave official documentation.

Finally we will focus on the canva farbling feature and how to attack it.

Plugins protection

Introduction

Here we will focus on the defense mechanisms used by the Brave browser to protect the list of used plugins announced by the browser to the website the user visits.

Brave implementation

The anti-fingerprinting protection system for plugins is straightforward. In *balanced* mode, the browser will add two fake plugins to the browser list of plugins, each with a name, a description and a filename randomly generated, with a length of 8, 32 and 16 characters respectively. The browser will then shuffle the list of plugins that it announces. The *maximum* mode will remove all the elements from the plugins list and generate two fake ones. In our experimentation, we focused on the *balanced* mode and on the implementation made in PR n°5989 [\[9\]](#), but it's important to note that parts of the process have changed since then, see [Limits](#) for more information.

Our implementation

Our implementation of the protection system for plugins is almost identical to the one from PR n°5989 [9], except for the context used as the key for the HMAC function which has been abstracted as the concatenation of the session number and the visited TLD.

Attack

Since Brave is an open-source browser, we know how the fingerprinting defence mechanism is implemented. Therefore, we can attack said mechanism by comparing two sets of plugins, and using specific criteria derived from the farbling mechanism to determine if both lists came from the same browser. Here are the criteria we kept, in order of verification in the code:

- If there are exactly two different plugins between the two lists. Since farbling adds two fake plugins, the difference between two fingerprints from the same instance on different websites should only be of 2 plugins.
- If the name of the plugin, its description and its filename measure exactly 8, 32 and 16 characters respectively. This is checked on both plugins found in the previous step.
- If none of the words composing the name and the description are words that can be found in the English dictionary. In the case of a fake plugin, both name and description will be random, and unlikely to form a real word.

A score is then attributed, according to the number of criteria that are verified by the compared plugins list.

Experimental procedure

To test if our attack can accurately recognize several farbled lists of plugins coming from the same browser, we implemented the following procedure:

We first load the FPStalker dataset [1], a dataset containing 15 000 real world fingerprints coming from different browsers and extract all the lists of plugins which we will use as our working dataset. We then select a random list of plugins among the latter and generate 10 farbled versions based on it, keeping the last one as a comparison point. We also generate and store 10 indexes (the *stored indexes*) and insert each farbled list at said indexes inside our dataset. After doing this, we shuffle the whole dataset.

Then, we loop over each element in our dataset and compare them with our comparison point kept earlier. If the comparison result is equal to 3, which means all the criteria described in [Attack](#) are met, then we store the index at which the list (the *stored indexes*) was found and increment a counter of similar lists.

Once all elements have been compared, we check if the *found indexes* are the same as the *stored indexes*. Because of the way the insertion works, the stored indexes might not be fully accurate. For example, if an element is inserted at index 10, and a second element is

inserted at index 5, then the first element is now at index 11. To palliate this problem, if a found index is within $[-10, +10]$ of a stored index, it is considered valid.

Finally, we display the number of similar lists found, i.e. lists that validate the 3 criteria of the attack, as well as the approximate number of original lists found, i.e. lists that validate the criteria and are the one we farbled in the first place.

Limits

The reimplementaion of the plugin protection system is based on a previous version of the process (PR n°5989 [9]). It has since been slightly modified [11], in particular by adding random changes of the words *Chrome*, *PDF*, *Plugin*, and *Viewer*, taken from a predetermined list; and changing the size of the name, description and filename randomly generated for the second fake plugin. However, implementing countermeasure to these defenses should be rather simple, albeit making the attack a little bit more costly.

This attack focuses on tracking users in a single browsing session, but not on tracking users across different sessions.

This attack cannot handle a change in the list of plugins (an addition, a deletion or a modification) beyond the farbling procedure. If a user were to add a plugin between two website visits, our attack would not be able to detect it.

The experimental procedure is a little bit imprecise, mainly due to the range of found indexes considered valid.

The *found indexes* are not tested to see if their respective fingerprints belong to the same user as the one from the original fingerprint chosen.

Discussion

Found indexes

During our testing, our attack found between 11 and 535 similar plugin lists, i.e. lists that validate all criteria of the attack, but are not necessarily part of the farbled lists we added. 11 lists are the minimum expected number that we can find, as the attack will find the 10 farbled lists, as well as the original list. Indeed, the difference between the original list and a farbled list will only be the two fake plugins, so the attack is still effective. In most cases we found more lists (12 and upwards). This is because a lot of fingerprints in the dataset come from users who were tracked for a long time and have therefore a lot of fingerprints associated to them. However, this caused some issues for the *stored indexes*, as having many *found indexes* increases the chance of one of them being in the $[-10, +10]$ range around the *stored index*, and we can observe that the more similar plugins are detected, the more identical plugins are reported.

This means that our attack is probably capable of linking a non-protected list of plugins with a farbled one, allowing tracking in case of a user going from *unprotected mode* to *balanced mode* in Brave, or changing browsers for Brave while keeping the same plugins.

Stored indexes found

Our attack consistently finds the 10 lists we farbled (± 2 elements), although it sometimes finds a little more, up to 10 more in our observations. This is the case when a lot of similar plugins lists are detected, as described in [Found indexes](#).

This means that our attack is capable of consistently defeating the fingerprinting defense for plugins lists.

Efficiency

In terms of performance, we used a dataset composed of 15 000 real life fingerprints, and while doing 10 sessions (for a total of 150 000 fingerprints processed), the total processing only took around 7 minutes on a commercial laptop (processor: Intel Core I7 11370H). This means that even with our naïve implementation, the processing of large quantities of fingerprints (or at least their list of plugins) is reasonably fast, and it is possible to imagine that even a threat actor with moderate resources could process huge quantities of fingerprints.

Conclusion

As we have seen, defeating the fingerprinting protection for the plugins used by Brave is quite easy, even with a naïve approach, making one able to track a user across different websites during a single browsing session. Our attack even allows us to recover previous fingerprints from a user that were not farbled, allowing tracking to persist even after enabling the defense mechanism. This attack, combined with similar ones on other elements of a browser fingerprint, might very well allow an attacker to track users' activity on the web even when a fingerprinting defense system is in place, which is a great privacy issue.

Protection through Fingerprint Uniformization

Objective

Uniformization is one of the mechanisms used by Brave Browser to mitigate browser fingerprinting. Instead of blocking access to APIs, which would be easily detectable and would break many websites, this approach reduces the variability of exposed attributes so that different browser users appear as similar as possible.

In this section, we test how well uniformization actually works. We want to see if lowering the precision of certain data is enough to stop re-identification, or if the remaining 'leaks' still allow for effective tracking

Methodology

Dataset and Assumptions

Our work is based on the FPStalker dataset [1], which contains real-world browser fingerprints. This dataset does not provide fingerprints from Brave users, so we rely on Chromium-based fingerprints from Google Chrome. This choice is justified by the fact that Brave is built on Chromium and exposes similar fingerprinting surfaces. We simulate brave's defenses by applying the exact same transformations that are in its standard mode [2]. This gave us a realistic, reproducible way to see if these protections are effective.

Simulation of Uniformization

Uniformization is implemented by reducing the precision of selected attributes and replacing highly discriminating values with more generic ones. In practice, graphics-related information exposed through Canvas and WebGL APIs is standardized to common identifiers. For the screen resolution we apply rounding to the nearest hundred (ex: 1440x900 becomes 1500x900), thereby creating anonymity sets. Language settings are simplified to retain only the primary language.

Importantly, not all attributes are modified. Values such as the User-Agent, timezone, and certain system-level identifiers remain unchanged (according to Brave documentation), altering them would bring compatibility issues. These attributes therefore constitute a residual source of entropy that may still enable user identification.

Evaluation Metrics

We evaluate the effectiveness of uniformization using two metrics.

- The uniqueness rate measures the proportion of fingerprints that remain unique within the dataset, it's our indicator of identifiability.
- Shannon entropy is used to quantify the amount of information contained in the fingerprints. A lower entropy indicates reduced distinguishing power.

Results

Effect of Uniformization

Experiments conducted on a subset of 483 fingerprints show that uniformization leads to a measurable reduction in identifiability. Before applying any transformation, 80.12% of users are uniquely identifiable, with an entropy of 8.655 bits. After uniformization, the uniqueness rate decreases to 65.22%, and entropy drops to 8.312 bits.

These results confirm that uniformization effectively reduces the entropy of individual attributes and increases the size of anonymity sets. However, the reduction remains limited, as most users still retain unique fingerprints after transformation.

Residual Uniqueness

To assess the robustness of this mechanism, we analyze the contribution of attributes that are not modified by Brave. These residual features, including the User-Agent, timezone, and operating system, remain accessible and retain significant identifying power.

Our results show that the combination of these attributes allows the unique identification of 36.85% of users, even after uniformization. The User-Agent alone contributes 13.87% to this uniqueness. This demonstrates that basic browser information, although not explicitly designed for tracking, can still support effective fingerprinting when combined.

Discussion

The results highlight a fundamental limitation of uniformization. While reducing the precision of individual attributes lowers their standalone's discriminating power, it does not eliminate the correlations between them. As a result, combining multiple partially protected attributes still enables user re-identification.

This limitation reflects a broader compromise in the design of fingerprinting defenses. Increasing the level of uniformization would require modifying additional attributes, which could negatively impact website compatibility and user experience. This constraint explains why certain high-entropy features remain unmodified in practice.

Brave developers came to this conclusion as they removed their strict mode, which made users more identifiable [\[3\]](#).

Conclusion

Uniformization reduces the identifiability of browser fingerprints by lowering the entropy of exposed attributes. Our experiments confirm that this approach is effective in decreasing both uniqueness and information content within a real-world dataset.

However, this reduction is not sufficient to prevent tracking. Residual attributes that are left unchanged continue to provide a strong basis for identification, especially when combined. These results suggest that uniformization alone cannot fully mitigate fingerprinting and must be complemented by additional protection mechanisms.

Targeted Feature: Randomization Canvas

The Brave browser uses a "Randomized Output" technique called **Farbling** to protect the Canvas API [\[1\]](#). It does not block the Canvas completely. Instead, it allows the website to read the Canvas, but it adds random noise to stop the website from creating a stable fingerprint.

How Farbling (for Canvas) works (Default mode):

- **Targeted selection:** The algorithm only changes a very small part of the image. It modifies a maximum of 256 pixels [\[10\]](#).
- **Minimal changes:** For each chosen pixel, Brave only changes one color channel (Red, Green, or Blue). This depends on the domain name of the website. The change is made by flipping the last bit (Least Significant Bit or LSB) using an XOR operation [\[2\]](#).
- **Noise generation:** The pixels are chosen pseudo-randomly. The "seed" (starting point) for this randomness is a key created from three things: the session ID, the website domain, and the original Canvas image itself.

Because of this, if a user visits the same website in two different sessions, the noise added to the Canvas will be different. This makes the fingerprint unstable.

Implementation: Dataset, Defense, and Attack

To show that this protection has a flaw, we coded a statistical attack using a "majority vote" method. We tested it on real data.

A. Dataset Extraction (FPStalker)

We used real fingerprints from the public FPStalker dataset [\[1\]](#) (extension1.txt). These were collected from real users on the internet. Because the file is a very large SQL database dump, we used the Pandas library and Regex (Regular Expressions) to easily find the Base64-encoded PNG images inside the file. Then, we decoded the image and turned it into a NumPy array (RGBA format) with 96,000 values. This perfect image is our "ground truth" (reference image) for the test.

B. Modeling the Defense (Farbling)

We wrote Brave's defense function in Python, following the official algorithm. For each new session, the function calculates a new key and applies the XOR noise to a maximum of 256 pixels on the original image.

C. The Attack Code (Majority Vote)

The attack works because the noise changes in every session, but 99% of the image stays exactly the same. The algorithm collects the N canvases from different sessions and stacks them on top of each other. It uses a majority vote method for each pixel and each color channel [\[10\]](#). For example, if the real value of a pixel is 150, and the noise changed it to 151 in only 1 out of 5 sessions, the function will choose 150 because it is the most common value. This removes the noise.

Experimental Results

In our test, we changed the number of sessions (N) collected by the attacker. We wanted to see how fast the majority vote could recreate the original Canvas (96,000 values).

- **Session 1 (N=1):** 256 errors. The fingerprint has the maximum amount of noise allowed by Brave. The precision is 99.73%.
- **Session 2 (N=2):** The number of errors drops to 16 (Precision: 99.98%).
- **Session 3 (N=3):** 11 errors left. These happen because of statistical ties (when there is no clear majority yet to make a decision).
- **Session 4 (N=4): 0 errors.** The precision reaches 100%. Mathematics confirms that we only need 4 canvases to use the majority vote and find the original values [\[10\]](#).
- **Sessions 5 to 10:** 0 errors. The success rate stays stable at 100%. The study [\[10\]](#) confirms that with 5 sessions, the chance of failure is almost zero (around 1.953×10^{-17}).

Analysis: The results show that the main weakness of this protection is that it adds too little noise. If an attacker can collect the Canvas from just 4 different sessions, the statistical attack completely breaks the defense. It restores the user's real hardware fingerprint at 100%.

Project conclusion

Through the three attacks we implemented, we showed that Brave's protections, while effective in principle, can still be bypassed in practice. The plugin attack highlights that farbling can leak patterns that allow linking fingerprints within a session. The uniformization study shows that even when attributes are simplified, enough residual information remains to re-identify a significant number of users. Finally, the canvas attack demonstrates that adding small amounts of noise is not sufficient when an attacker can aggregate multiple observations.

Taken together, these results illustrate how difficult it is to truly protect against fingerprinting. Each mechanism reduces the attack surface, but none of them is enough on its own, and combining multiple signals often restores enough uniqueness for tracking.

This leads to a broader question: how far can browsers go in protecting users without breaking the web? Stronger protections, like those used in Tor Browser, come at the cost of a degraded browsing experience. In contrast, Brave Browser chooses a compromise, aiming

to improve privacy while remaining usable for the public. Our results suggest that this trade-off, while necessary, leaves room for attackers to exploit the remaining information.

References

- [1]: FPStalker dataset: <https://github.com/Spirals-Team/FPStalker>
- [2]: Brave's fingerprinting documentation: <https://github.com/brave/brave-browser/wiki/Fingerprinting-Protections>
- [3]: <https://brave.com/privacy-updates/3-fingerprint-randomization/>
- [4]: <https://brave.com/privacy-updates/28-sunsetting-strict-fingerprinting-mode/>
- [5]: Fingerprinting tools :
- [6]: AmlUnique website: <https://amiunique.org/>
- [7]: <https://fingerprintjs.github.io/fingerprintjs/>
- [8]: Canvas fingerprinting: https://en.wikipedia.org/wiki/Canvas_fingerprinting
- [9]: PR 5989 implementing plugin farbling: <https://github.com/brave/brave-core/pull/5989>
- [10]: *Analysis of Brave's Canvas Farbling*: <https://dl.acm.org/doi/abs/10.1145/3696410.3714713>
- [11]: Brave plugin farbling: https://github.com/brave/brave-core/blob/master/chromium_src/third_party/blink/renderer/modules/plugins/dom_plugin_array.cc